Review: fork() system call

- The entire virtual address space of the parent is replicated in the child
- At the time of fork() both memory spaces have the same content.
- The child process and the parent process run in separate memory spaces.
- The child inherits copies of the parent's set of open file descriptors.
  - Each file descriptor in the child refers to the same open file description (see open(2)) as the corresponding file descriptor in the parent.
  - This means that the two file descriptors share open file status flags, file offset, and signal-driven I/O attributes

Review: fork() system call

- **On success,**
  - returns PID of the child process in the parent,
  - returns 0 in the child.

- **On failure,**
  - -1 is returned in the parent,
  - no child process is created,
  - and **errno** is set appropriately.
  - **EAGAIN** A system-imposed limit on the number of threads was encountered. There are a number of limits that may trigger this error:
    - the kernel's system-wide limit on the number of processes and threads, **/proc/sys/kernel/threads-max,** was reached
    - the maximum number of PIDs, **/proc/sys/kernel/pid_max**, was reached
    - For more see `$man -S2 fork`

Fork and file descriptors

★ **File descriptors are copied in fork!**

```
/* one process reads one part
the other reads another part
*/
int file = open(...);
if(!fork()) {
   read(file, ...);
} else {
   read(file, ...);
}
```

```
/*we have two file
descriptors*/
if(!fork()) {
   int file = open(...);
   read(file, ...);
} else {
   int file = open(...);
   read(file, ...);
}
```

Review: wait() system calls

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

```
#include <sys/wait.h>
int WIFEXITED (status);
int WIFSIGNALED (status);
int WIFSTOPPED (status);
int WIFCONTINUED (status);
int WEXITSTATUS (status);
int WTERMSIG (status);
int WSTOPSIG (status);
int WCOREDUMP (status);
```

```
/*example use*/
wait(&status);
waitpid(-1, &status, 0);

/*more precise control: siginfo_t is a structure with pid, uid,status etc*/
int waitid(idtype_t idtype, id_t id,
     siginfo_t *infop, int options);
```

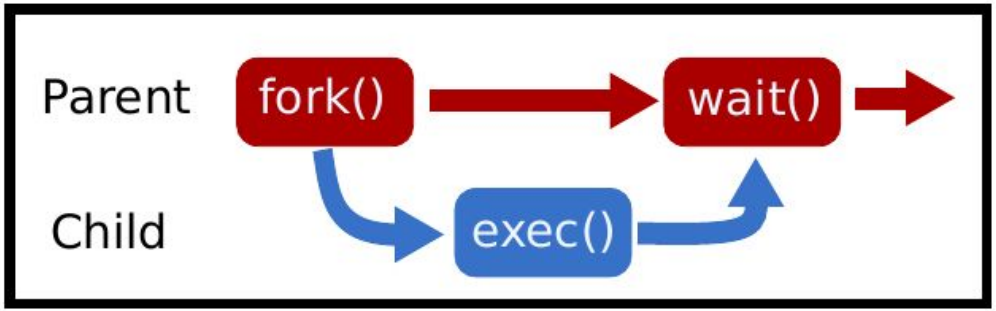
See $man -S2 wait

```c
/*zombie process
example
*/
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t cpid = fork();
    if(cpid > 0){
        printf("parent id: %d, child id = %d.\n", getpid(),  cpid);
        /*in terminal $ps aux |grep "defunct"
        see the ZOMBIE process*/
        getchar();/*wait for user input*/
        pid_t w = wait(NULL);
        if (w == -1){
            perror("wait:"); exit(EXIT_FAILURE);
        }else{
            printf("parent id: %d, ZOMBIE child id = %d.\n",
                getpid(), w);
           exit(EXIT_SUCCESS);
        }
    }else if(cpid == -1){
        perror("fork:");
        exit(EXIT_FAILURE);
    }
}
```

fork-exec-wait

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main() {
   pid_t pid = fork();
   if (pid < 0) {/* fork failure*/
       exit(1);
   } else if (pid > 0) {
       waitpid(pid, NULL, 0);
   } else {
       execl("/bin/ls", "/bin/ls", NULL);
       exit(1); /*For safety.*/
   }
}
```



Environment variables

Environment variables are variables that the system keeps for all processes to use.

Your system has these set up right now! In Bash, some are already defined

```
$ echo $HOME

$ echo $PATH
```

Environment variables are important because they are inherited between processes and can be used to specify a standard set of behaviors

```
$ echo $HOME

$ echo $PATH
```

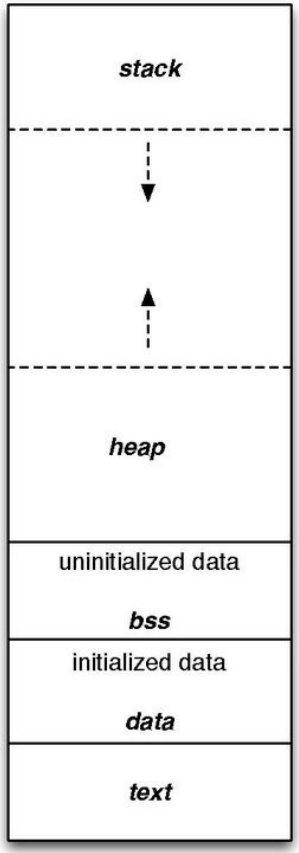
In C;

```c
#include <stdlib.h>
char* getenv (const char *name)
int setenv (const char *name, const char *value, int overwrite)
int putenv (const char *string)
void unsetenv (const char *name)
```

```c
/*example use*/
char* home = getenv("HOME");
setenv("HOME", "/home/adaskin", 1);
```

Environment variables are important because they are inherited between processes and can be used to specify a standard set of behaviors

Basic Memory Allocators



Memory allocation/deallocation is done by almost any application

The heap at the system level is contiguous series of addresses that the program can expand or contract

In LINUX, this is called the **program break**.

`brk()` and `sbrk()` change the location of the **program break**

C memory allocation API

```c
#include <stdlib.h>
/*Allocate SIZE bytes of memory.uninitialized!*/
void *malloc(size_t size);
```

```c
int *ptr;
ptr = (int*) malloc (sizeof(int));

if(ptr == NULL) abort();

memset(ptr, 0, sizeof(struct foo));
```

examples

```c
struct foo *ptr;
ptr = (struct foo *) malloc (sizeof(struct foo));

if(ptr == 0) abort();

memset(ptr, 0, sizeof(struct foo));
```

```c
int  *ptr[10]; /*10 pointers for integer-addresses*/

ptr[0] = (int *) malloc(sizeof(int);
ptr[1] = (int *) malloc(sizeof(int);
```

```c
int  **ptr; /* a pointer to for the address of apointer*/
ptr = (int **)malloc(sizeof(int*));
```

C memory allocation API

```c
/* Allocate NMEMB elements of SIZE bytes each,
all initialized to 0.*/
void *calloc(size_t nmemb, size_t size);
```

```c
int *ptr;
ptr = (int*) calloc(1000, sizeof(int));

if(ptr == NULL){
  perror("calloc:");
  exit(EXIT_FAILURE);
}
```

A simple Implementation of calloc()

```
void *calloc(size_t count, size_t eltsize){
  size_t size = count * eltsize;
  void *value = malloc(size);
  if (value != 0)
      memset(value, 0, size);
  return value;
}
```

Changing the size of the allocated memory

```
void *xrealloc(void *ptr, size_t size){
  void *value = realloc(ptr, size);
  if (value == 0)
      perror("Virtual memory exhausted");
  return value;
}
```

free()

```
int *ptr = malloc(sizeof(*ptr));
do_something(ptr);
free(ptr);
```

Usually, allows a later call to malloc to reuse the space.

In the meantime, the space remains in your program as part of a free-list used internally by malloc.

Occasionally, free can actually return memory to the operating system and make the process smaller.

Any use of a pointer that refers to freed space results in undefined behavior.

free()

Freeing a block alters the contents of the block. Copy whatever you need out of the block before freeing it!

```
struct chain{
  struct chain *next;
  char *name;
};
```

```
void free_chain(struct chain *chain){
  while (chain != 0)   {
      struct chain *next = chain->next;
      free(chain->name);
      free(chain);
      chain = next;
  }
}
```

realloc(): changes the size of the memory block to `newsize` bytes.

```
void * realloc(void * ptr, size_t newsize) {
 /*Simple implementation always reserves more memory  and has no error checking*/

 void *result = malloc(newsize);
 size_t oldsize = ... /*(depends on allocator's internal data*/
 if (ptr){
   memcpy(result, ptr, newsize < oldsize ? newsize : oldsize);
 }
 free(ptr);
 return result;
}
```

```c
void * realloc(void * ptr, size_t newsize) {
 /*Simple implementation always reserves more memory  and has no error checking*/

 void *result = malloc(newsize);
 size_t oldsize = ... /*(depends on allocator's internal data*/
 if (ptr){
   memcpy(result, ptr, newsize < oldsize ? newsize : oldsize);
 }
 free(ptr);
 return result;
}
```

If realloc fails then the program **leaks memory.**

```c
int main() {
 /* 1 */
 int *array = malloc(sizeof(int) * 2);
 array[0] = 10; array[1] = 20;

 /* Oops need a bigger array -
     so use realloc..*/
 array = realloc(array, 3 * sizeof(int));
 array[2] = 30;
}
```

Memory leaks

More robust version

```
int main() {
 int *array = malloc(sizeof(int) * 2);
 array[0] = 10; array[1] = 20;

 void *tmp = realloc(array, 3 * sizeof(int));

 if (tmp == NULL) {  /*Nothing to do here.*/

 }else if (tmp == array) {/*realloc returned same space*/
   array[2] = 30;

 }else {/* realloc returned different space */
   array = tmp;
   array[2] = 30;
 }
}
```

Heap and sbrk() system call

- The heap is part of the process memory and varies in size.
- Heap memory allocation is performed by the C library when a program calls malloc (calloc , realloc) and free .

- By calling **sbrk()** the C library can increase the size of the heap as your program demands more heap memory.

Modern operating system memory allocators no longer need sbrk.

Instead, they can request independent regions of virtual memory and maintain multiple memory regions (mmap).

Sbrk may be used for small memory requests

Changing the location of the program break

```
<unistd.h>
int brk (void *end_data_addr)
```

sets the end of the calling process' data segment to `end_data_addr`.

```
void *sbrk (intptr_t increment);
```

Increases the current end by `increment`

`sbrk(0)` returns the current location of the program break

On error, errno is set to ENOMEM

Notes on sbrk

Program break: the end of the process's data segment (i.e., the program break is the first location after the end of the uninitialized data segment).

Programs don’t need to call **brk** or **sbrk** typically.

Instead programs use **malloc , calloc , realloc** and **free** which are part of the C library.

- The internal implementation of these functions may call **sbrk** or other system calls (such as **mmap**) when additional heap memory is required.

```c
void *top_of_heap = sbrk(0);

sbrk(1024);

malloc(16384);/implementation is different on different systems (virtual memory etc)*/

void *top_of_heap2 = sbrk(0);
printf("The top of heap went from %p to %p \n", top_of_heap,top_of_heap2);
```

- **The memory that was newly obtained by the operating system must be zeroed out (otherwise security leak).**
- **But, later calls are not always zero.**

```c
char* ptr = malloc(300);
/*contents is probably zero because we get brand new memory so beginner
programs appear to work!*/

//strcpy(ptr, "Some data"); /*work with the data*/
free(ptr);



/* later*/
char *ptr2 = malloc(300);
/*Contents might now contain existing data and is probably not zero*/
```

Implementing malloc() and free()

the simplest implementation of malloc

- Ask the system for more bytes by extending the heap space.

sbrk returns -1 on failure

```c
void *malloc(size_t size){
    void *p = sbrk(size);
   if (p == (void *)-1) /* No space left*/
       return NULL;
   return p;
}
void free(){ /* Do nothing */
}
```

Drawbacks:

- System calls are slow compared to library calls.
  - We should reserve a large amount of memory and only occasionally ask for more from the system.
- **No reuse of freed memory.**
  - Our program never re-uses heap memory - it keeps asking for a bigger heap.

Reusing a memory

During program execution, memory is allocated and deallocated

- so there will be a gap in the heap memory that can be re-used for future memory requests.
- The memory allocator needs to keep track of which parts of the heap are currently allocated and which are parts are available.

Free space management

- How to implement?
  - An array, list?
  - How to keep track of free/not-free?
  - Placement strategy?

Placement Strategies

Figure 5.1: Empty heap blocks

```
malloc(2048)
```

- where should malloc reserve the memory?
  - First fit strategy
    - Next fit
  - Best fit
  - Worst fit

Placement Strategies

A first-fit strategy finds the first available hole that is of sufficient size so break the 16KiB hole into two.

We don’t have to look through the entire heap!

Next fit starts where it is left off in the previous allocation instead of starting from the beginning of the heap.

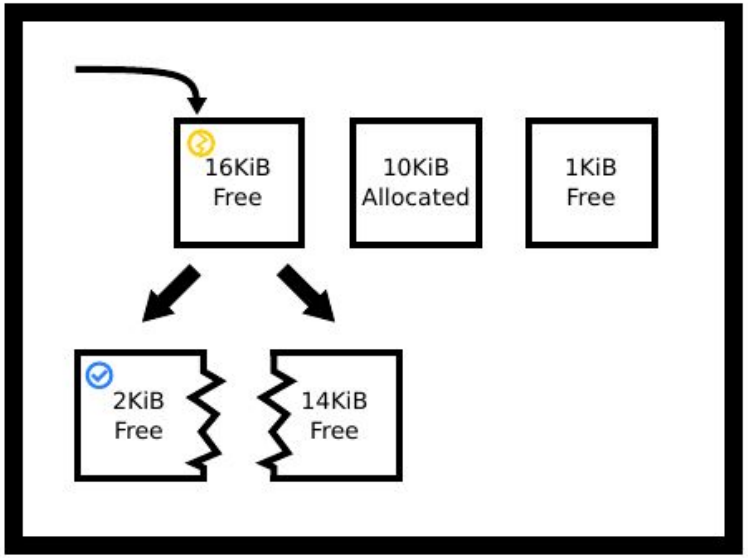


Figure 5.4: First fit finds the first match

Placement Strategies

A **perfect-fit strategy (best-fit)** finds the smallest hole that is of sufficient size (at least 2KiB)

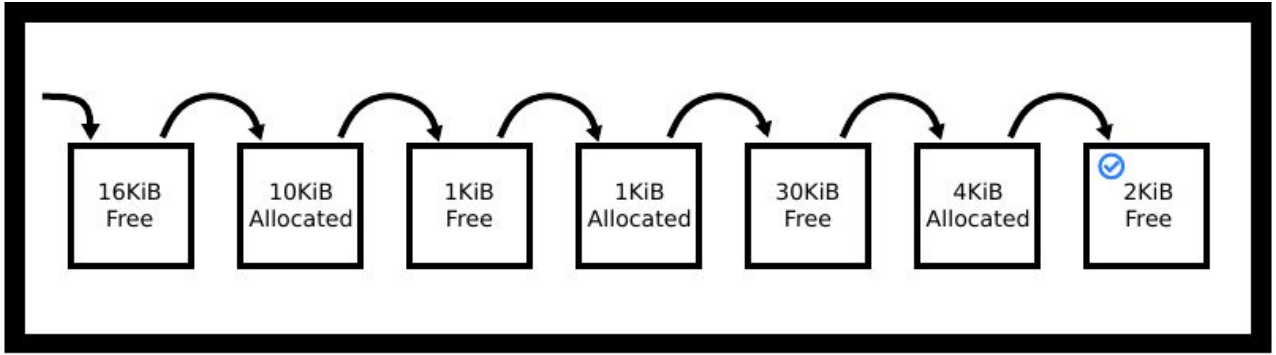


Figure 5.2: Best fit finds an exact match

Placement Strategies

A **worst-fit strategy** finds the largest hole that is of sufficient size so break the 30KiB hole into two:

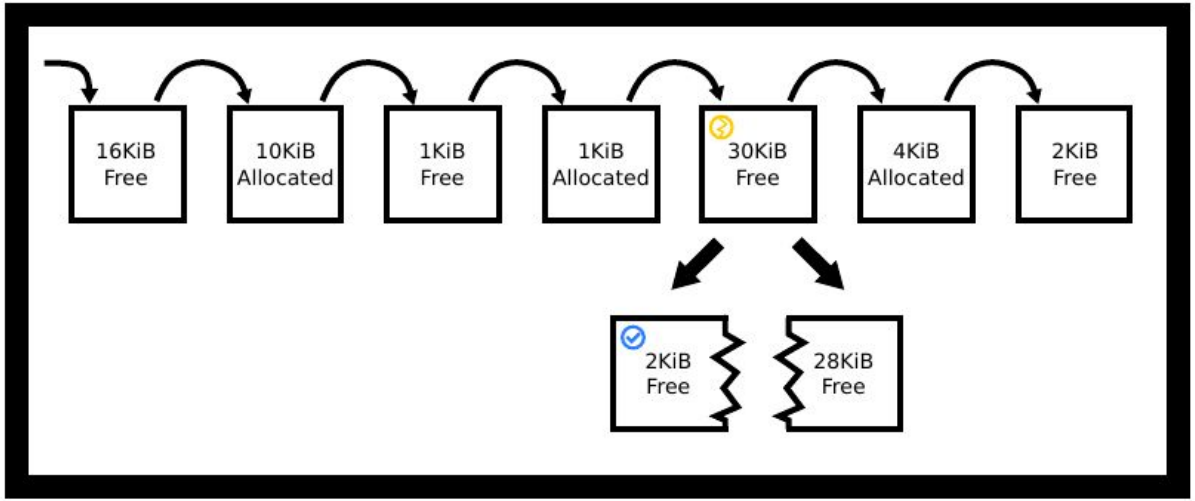


Figure 5.3: Worst fit finds the worst match

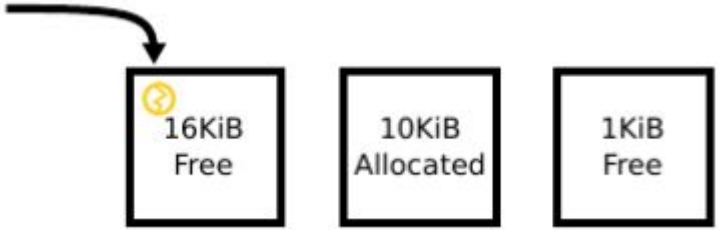
Fragmentations

internal fragmentation

Example:

First fit allocator could have returned the original 16KiB block unbroken.

- User wants 2KiB
- Malloc returns 16KiB
  - 2KiB used by the user
  - **14KiB unused** by the user and the allocator.



The unused **14KiB** is **internal fragmentation.**

Fragmentations

external fragmentation

even though we have enough memory in the heap, but not continuous!

- it is divided up in a way so there is no continuous block of required memory.

Example we have 39KiB free memory, but cannot allocate 32KiB

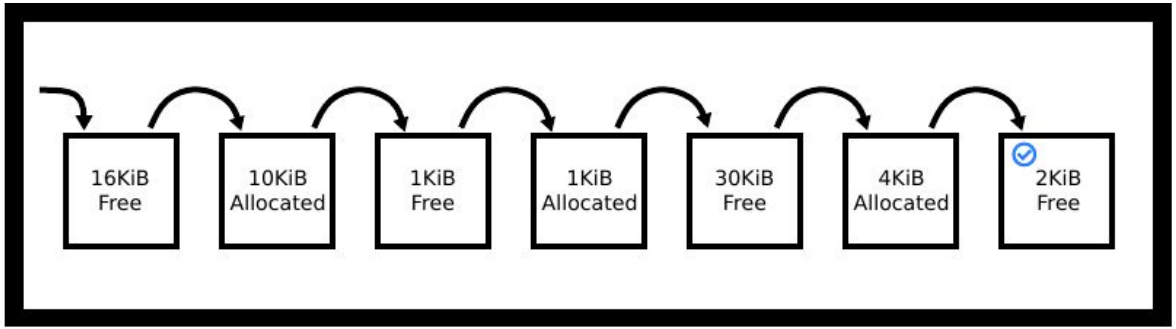


Figure 5.2: Best fit finds an exact match

Note: Anonymous memory mapping through mmap resolves external fragmentation.

The challenges of writing a heap allocator

- Need to minimize fragmentation (i.e. maximize memory utilization)
- Need high performance
- Fiddly implementation – lots of pointer manipulation using linked lists and pointer arithmetic.

The challenges of writing a heap allocator

- Both fragmentation and performance depend on the application allocation profile,
  - this can be evaluated but not predicted and in practice,
  - under-specific usage conditions, a special-purpose allocator can oftenout-perform a general-purpose implementation.
- The allocator doesn’t know the program’s memory allocation requests in advance.
- Even if we did, this is **the Knapsack problem** which is known to be **NP-hard**!

Placement Strategy Pros and Cons

Theoretical Analysis: The problem of bin packing.

Given a list A = (α_1,α_2,...α_n,) of real numbers in the range (0, 1], place these in a minimum number of “bins” so that no bin holds numbers summing to more than 1.

let A* be the smallest number of bins into which the numbers of list may be placed.

- First fit: worst case 1.7
- Best fit: at least 1.7

But, a few real-world apps need this type of one-shot allocation.

https://dl.acm.org/doi/pdf/10.1145/800152.804907
M. R. Garey, R. L. Graham, and J. D. Ullman. Worst-case analysis of memory allocation algorithms. In Proceedings of the Fourth Annual ACM Symposium on Theory of Computing, STOC ’72, pages 143–150, New York, NY, USA, 1972. ACM. http://doi.acm.org/10.1145/800152

Placement Strategy Pros and Cons

Some practical remarks:

Best fit may take less time than a full heap scan. When a block of perfect size or of perfect size within a **threshold** is found, that can be returned, depending on what edge-case policy you have.

Worst fit follows this as well. Your heap could be represented with the max-heap data structure and each allocation call could simply pop the top off, reheapify, and possibly insert a split memory block.

First fit needs to have a block ordering (e.g. linked lists). with address ordered linked lists you can speed up insertion from O(n) to O(log(n)) by using a randomized skip-list in conjunction with the linked list.

Implementing a naive-memory allocator

- A memory allocator needs to keep track of which bytes are currently allocated and which are available for use.
- Our heap memory is a list of blocks where each block is either allocated or unallocated.
- We will create linked lists to keep track of blocks
  - **Implicit free list:** We can find the next block by finding the end of the current one
    - We use math to find the next block
  - Or we can use **Explicit free list**

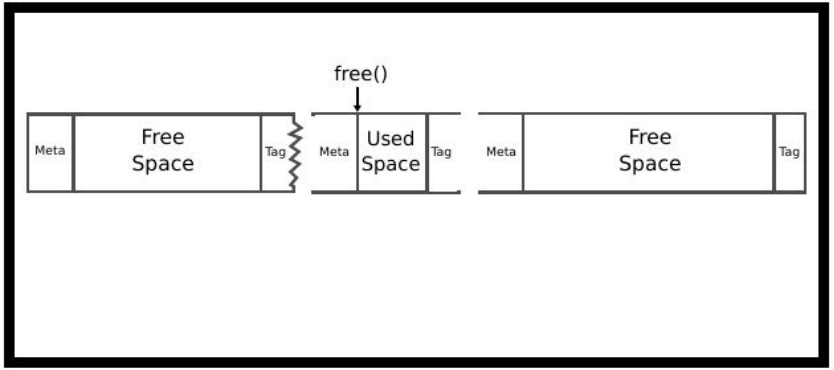


Figure 5.5: 3 Adjacent Memory blocks

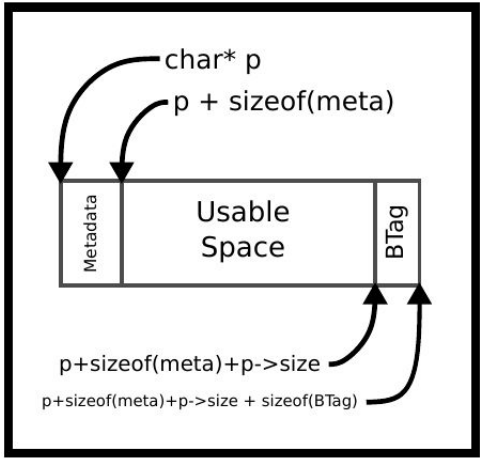


Figure 5.6: Malloc addition

Implicit free list implementation

```
typedef struct {
 size_t block_size;
 char data[];
} block;
```

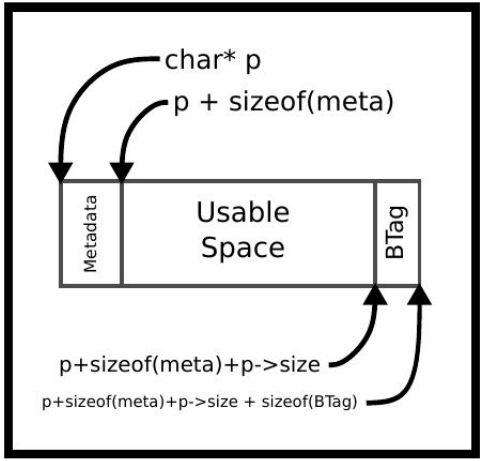


Figure 5.6: Malloc addition

```
block *p = sbrk(100);
p->block_size = 100 - sizeof(metadata) - sizeof(BTag);

/* Other block allocations*/
```

Navigation in implicit free list

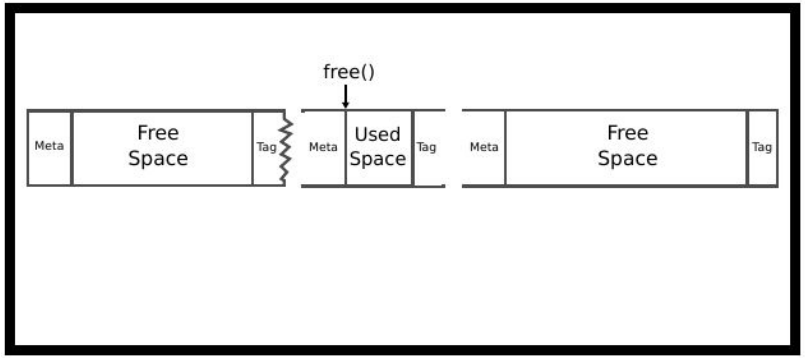


Figure 5.5: 3 Adjacent Memory blocks

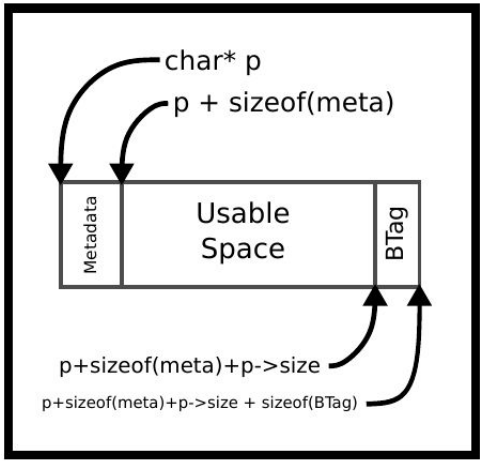


Figure 5.6: Malloc addition

```
/*We could navigate
from one block to the next block
by adding the block's size.*/
p + sizeof(metadata) + p->block_size + sizeof(BTag)
```

The simplest implementation uses first fit.

Start at the first block, assuming it exists, and iterate until a block that represents unallocated space of sufficient size is found, or we've checked all the blocks.

If no suitable block is found, it's time to call sbrk() again to sufficiently extend the size of the heap.

```
typedef struct {
  int is_free;
  size_t block_size;
  char data[];
} block;
block *p = sbrk(100);
p->block_size = 100
                - (sizeof(metadata)
                + sizeof(btag));
```

```
typedef struct {
   unsigned int is_free : 1;
   unsigned int block_size : 31;
} size_free;
typedef struct {
   size_free info;
   char data[];
} block;
```

Split & alignment

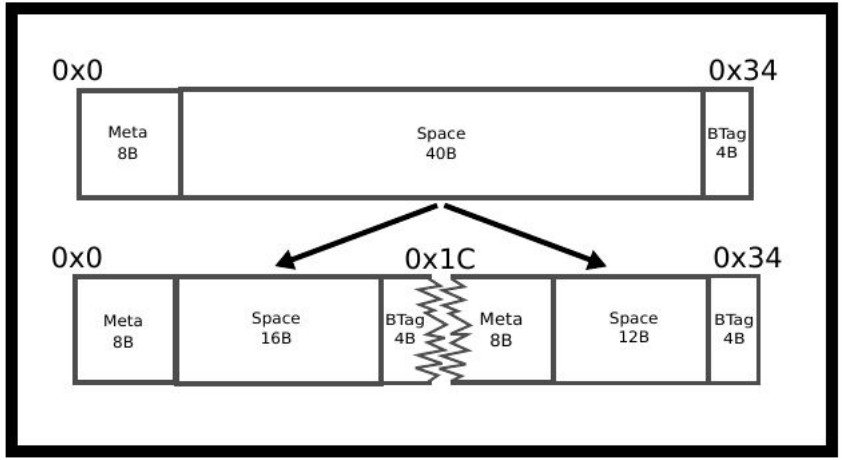


Figure 5.7: Malloc split

Alignment: Many architectures expect multibyte primitives to be aligned to some multiple of 2 (4, 16, etc).

- Address of K-byte primitive data type must be multiple K
- For example, it's common to require 4-byte types to be aligned to 4-byte boundaries and 8-byte types on 8-byte boundaries.

If multi-byte primitives are stored on an unreasonable boundary, the performance can be significantly impacted because it may require an additional memory read.

On some architectures the penalty is even greater - the program will crash with a bus error.

Split & alignment

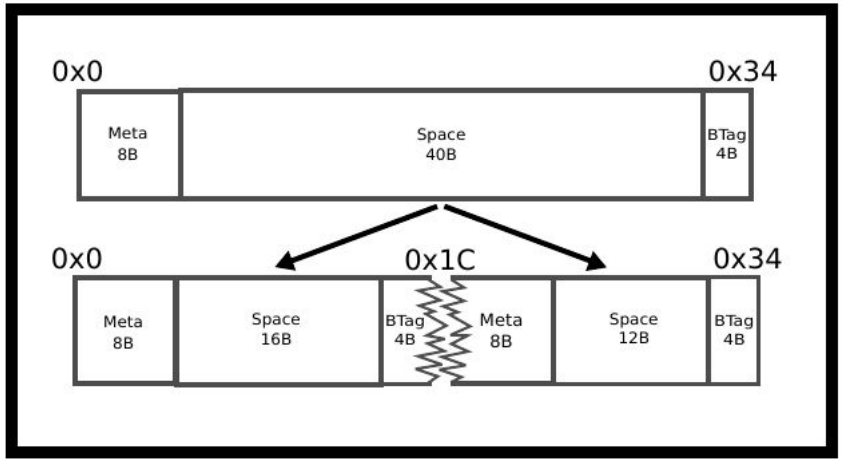


Figure 5.7: Malloc split

```
/*number of 16 byte-units*/
int s = (requested_bytes + tag_overhead_bytes + 15) / 16
```

Implementing Free

```
p->info.is_free = 0;
```

We need to do more!

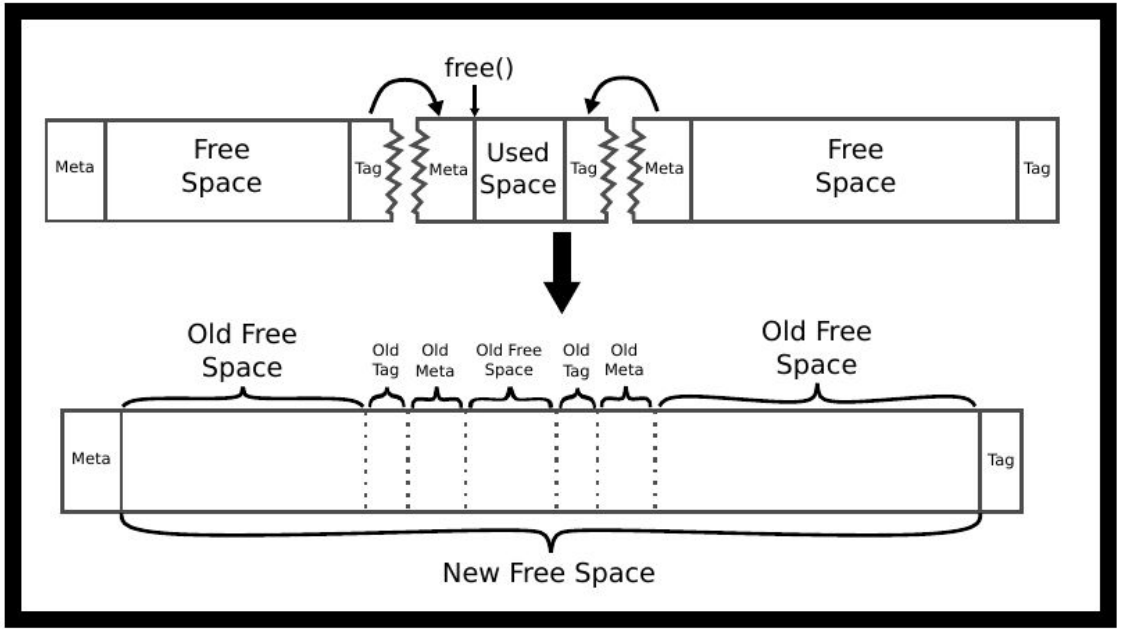
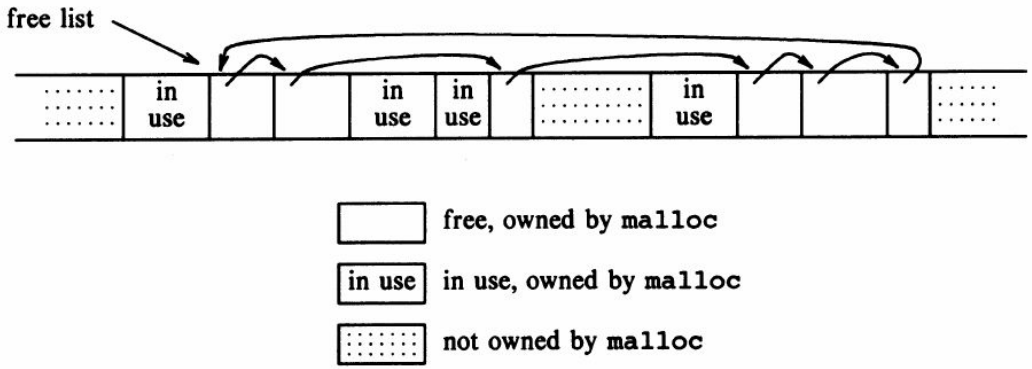
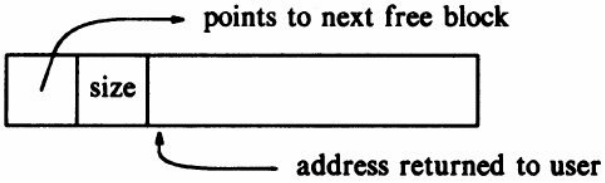


Figure 5.8: Free double coalesce

Explicit free lists allocators for better performance

List of only free blocks!





A block returned by malloc

Where do we put newly freed block?

```
struct {
  size_t info;/*number of  16 byte blocks*/
  struct block *next;
  char data[];
}block;
```

- **LIFO:** insert freed block at the beginning
  - Simple and constant time
  - Studies suggest fragmentation is worse
- **Address-ordered policy:** insert blocks so that blocks are always in address order
  - Requires linear time search with regular linked lists
    - Logarithmic time with the use of balanced trees
  - Studies suggest ft fragmentation is better

Coalescing in explicit free lists

Neighboring free blocks are already part of the free list

- Remove old block
- Add new coalesced block

How do we tell neighboring blocks are free?

- Use boundary tags

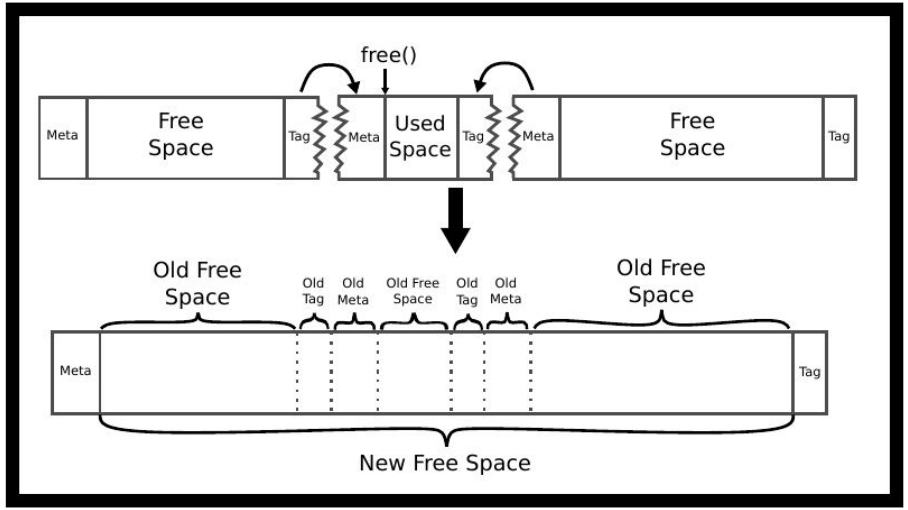


Figure 5.8: Free double coalesce

Segregated Lists

Segregated Lists

Keep separate lists of blocks for different sizes:

E.g.: list of 8s, list 16s, list of 32s, list of 64 …

Suballocators for each free list

Buddy Allocator

Block size: 2^n, n=1,2,3...

If there are no free blocks of 2^n, go to the next level and steal that block and split into two

If two neighboring blocks of the same size free, coalesce into large block of twice the size

- Main disadvantage: internal fragmentation
  - 68-byte requires 128-byte block

Slab allocator: fixed sizes for special data structures

Summary of keeping track of free blocks

- Implicit list: using sizes to determine next block
- Explicit list: linked list amont free blocks by using pointers
- Segregated list: different free list for different sizes

- You can also sort blocks by size
  - Use balanced tree, length is used as the key

sbrk() vs mmap()

sbrk()&brk()

No longer part of POSIX,

used in malloc for small memory allocations

Easy to extend the heap memory

Not easy to hand back the allocated memory to OS

mmap()

Allocate some resource into the process's memory space (memory mapped files)

Process's address space is composed of different memory areas

Easy to allocate and deallocate(hand back to OS)

POSIX standard, used in malloc() for large blocks